Playwright with Typescript interview questions

SECTION - A
(BASICS TS & PW)

- 🔷 Basic TS questions
- 🔷 Basic pw questions
- 🔷 Basic Scenarios in PW

Basic TS questions

🟢 **1. What is TypeScript and how does it differ from JavaScript?**

✅ **Answer:**

TypeScript is a **superset of JavaScript** developed by Microsoft that adds **static typing and compile-time checking**.

👉 It gets compiled into JavaScript using the TypeScript compiler (tsc).

🔍 **Key Differences:**

Feature	TypeScript	JavaScript
Typing	Static	Dynamic
Error detection	Compile-time	Runtime
OOP support	Strong	Limited
Maintainability	High	Medium

💡 **Example:**

```
let name: string = "Yugandhar";
name = 10;  // ❌ Error in TypeScript

let name = "Yugandhar";
name = 10;  // ✅ Allowed in JavaScript
```

🎯 **Interview Line:**

Playwright with Typescript interview questions

“TypeScript improves code reliability by catching errors at compile time, unlike JavaScript which fails at runtime.”

🟢 **2. Why do we use TypeScript in automation frameworks?**

✅ **Answer:**

TypeScript is used to make automation frameworks:

- More **stable**
- More **maintainable**
- Less **error-prone**

🔍 **Benefits in Automation:**

1. **Type Safety**
    - Prevents wrong data usage
2. **Better IntelliSense**
    - Auto suggestions in IDE
3. **Refactoring Support**
    - Easy to update large frameworks
4. **Early Error Detection**
    - Saves execution time

💡 **Example:**

```
function login(username: string, password: string) {}
login(123, true);  // ❌ Type error
```

🎯 **Interview Line:**

“TypeScript ensures robust and scalable automation frameworks by enforcing type safety and reducing runtime failures.”

🟢 **3. What are primitive data types in TypeScript?**

Playwright with Typescript interview questions

✅ **Answer:**

Primitive types are **basic data types** used to store simple values.

🔷 **Common Primitive Types:**

- string
- number
- boolean
- null
- undefined
- void
- bigint
- symbol

💡 **Example:**

let name: string = "Test";
let age: number = 25;
let isActive: boolean = true;

🎯 **Interview Line:**

“Primitive types represent single values and are the foundation of TypeScript typing system.”

🟢 **4. Difference between any, unknown, and never**

✅ **any**

let value: any = "hello";
value = 10; // ✅ allowed

👉 No type checking ❌ (dangerous)

✅ **unknown**

let value: unknown = "hello";

if (typeof value === "string") {

```
 console.log(value.toUpperCase());
}
```

👉 Safer than any ✓

✅ **never**

```
function error(): never {
 throw new Error("Error occurred");
}
```

👉 Represents:

- No return
- Impossible value

🎯 **Interview Line:**

“any disables type safety, unknown enforces validation, and never represents values that never occur.”

🟢 **5. What is a variable and how do you declare it?**

✅ **Answer:**

A variable is a **container to store data** with a specific type.

💡 **Syntax:**

```
let name: string = "Test";
```

🔷 **Ways to declare:**

```
let a: number = 10;   // type + value
let b: number;        // type only
let c = 10;           // inferred type
```

🎯 **Interview Line:**

“Variables in TypeScript are strongly typed containers used to store values safely.”

Playwright with Typescript interview questions

🟢 6. Difference between let, const, and var

Feature	let	const	var
Scope	Block	Block	Function
Reassign	Yes	No	Yes
Hoisting	No	No	Yes

💡 **Example:**

let a = 10;
a = 20; // ✅

const b = 10;
b = 20; // ❌

var c = 10;

🎯 **Interview Line:**

“let and const are block-scoped and safer, while var is function-scoped and generally avoided.”

🟢 7. What is type inference?

✅ **Answer:**

TypeScript automatically detects the type based on assigned value.

💡 **Example:**

let name = "Yugandhar"; // inferred as string
name = 10; // ❌ error

🎯 **Interview Line:**

“Type inference allows TypeScript to automatically determine variable types without explicit annotations.”

Playwright with Typescript interview questions

🟢 8. What are union types?

✅ **Answer:**

Union types allow a variable to hold **multiple types**.

💡 **Example:**

```
let value: string | number;

value = "hello";
value = 10;
```

🎯 **Interview Line:**

“Union types provide flexibility while maintaining type safety.”

🟢 9. How do you handle null and undefined?

✅ **Answer:**

Use strict checks and optional chaining.

💡 **Example:**

```
let name: string | null = null;

if (name !== null) {
  console.log(name.toUpperCase());
}
```

💡 **Optional chaining:**

```
user?.name
```

🎯 **Interview Line:**

“We handle null and undefined using strict checks and optional chaining to avoid runtime errors.”

🟢 10. What is an interface?

Playwright with Typescript interview questions

✅ **Answer:**

An interface defines the **structure of an object**.

💡 **Example:**

```
interface User {
  name: string;
  age: number;
}

const user: User = {
  name: "Test",
  age: 25
};
```

🎯 **Interview Line:**

“Interfaces define contracts that objects must follow.”

🟢 **11. Difference between interface and type**

Interface	Type
Extendable	Flexible
OOP focused	Functional
Declaration merging	No merging

🎯 **Interview Line:**

“Interfaces are preferred for object structures, while types are more flexible for unions and complex types.”

🟢 **12. What are enums?**

✅ **Answer:**

Enums are used to define **constant values**.

💡 **Example:**

```
enum Status {
  SUCCESS,
  FAIL
}

let result: Status = Status.SUCCESS;
```

🎯 **Interview Line:**

“Enums provide meaningful names to constant values, improving readability.”

🟢 **13. How do you define an array?**

💡 **Example:**

```
let numbers: number[] = [1, 2, 3];

let names: Array<string> = ["a", "b"];
```

🎯 **Interview Line:**

“Arrays in TypeScript can be defined using type[] or Array<type> syntax.”

🟢 **14. What is void?**

✅ **Answer:**

void means **no return value**

💡 **Example:**

```
function log(): void {
  console.log("Hello");
}
```

🎯 **Interview Line:**

“Void is used when a function does not return any value.”

Playwright with Typescript interview questions

🟢 **15. What is tsconfig.json?**

✅ **Answer:**

It is the **configuration file for TypeScript compiler**

💡 **Example:**

```
{
  "compilerOptions": {
    "target": "ES6",
    "strict": true,
    "module": "commonjs"
  }
}
```

🔍 **Purpose:**

- Controls compilation
- Enables strict checks
- Defines project structure

🎯 **Interview Line:**

“tsconfig.json defines how TypeScript code is compiled and ensures consistency across the project.”

Basic pw questions

🟢 **1. What is Playwright and who developed it?**

✅ **Answer**

Playwright is an **open-source end-to-end testing framework** for automating web browsers, developed by Microsoft.

🔍 **What it’s used for**

Playwright with Typescript interview questions

- UI (E2E) testing
- Cross-browser testing
- API testing (built-in request client)

🎯 **Key features**

- Auto-waiting
- Fast, reliable execution
- Headless/headful modes
- Powerful debugging (Trace Viewer)

💡 **Example**

```
import { test } from '@playwright/test';

test('open homepage', async ({ page }) => {
  await page.goto('https://example.com');
});
```

🟢 **2. What browsers does Playwright support?**

✅ **Answer**

- **Chromium** (Chrome, Edge)
- **Firefox**
- **WebKit** (Safari engine)

🎯 **Why important**

You can validate your app across **all major engines** with one framework.

🟢 **3. What languages does Playwright support?**

✅ **Answer**

- TypeScript
- JavaScript
- Python
- Java
- .NET (C#)

🎯 **Interview tip**

“TypeScript is preferred due to type safety and better IntelliSense.”

Playwright with Typescript interview questions

🟢 4. Why Playwright over Selenium?

✅ Answer (comparison)

Feature	Playwright	Selenium
Waits	Auto-wait	Manual waits
Speed	Faster	Slower
Setup	No WebDriver	Needs drivers
API Testing	Built-in	Separate tools
Architecture	WebSocket/CDP	HTTP

🎯 Interview line

“Playwright is faster, more reliable, and reduces flakiness due to built-in auto-waiting and modern browser control.”

🟢 5. How do you install and initialize Playwright?

✅ Step-by-step

```
npm init playwright@latest
```

🔍 What it does

- Installs Playwright
- Sets up project structure
- Installs browsers
- Creates sample tests

💡 Manual install

```
npm install -D @playwright/test
npx playwright install
```

🟢 6. What is Playwright Test Runner?

✅ Answer

Playwright Test Runner is a **built-in test framework** for writing and executing tests.

🔍 Features

Playwright with Typescript interview questions

- Parallel execution
- Fixtures
- Retries
- Reporting
- Assertions (expect)

💡 **Example**

```
import { test, expect } from '@playwright/test';

test('title test', async ({ page }) => {
  await page.goto('https://example.com');
  await expect(page).toHaveTitle(/Example/);
});
```

🟢 **7. What is browser, context, and page?**

✅ **Answer**

Component	Meaning
browser	Entire browser instance
context	Incognito session
page	Single tab

💡 **Example**

```
const browser = await chromium.launch();
const context = await browser.newContext();
const page = await context.newPage();
```

🎯 **Interview line**

“Browser is the engine, context is an isolated session, and page represents a tab.”

🟢 **8. How do you launch a browser?**

💡 **Example**

```
import { chromium } from '@playwright/test';
```

Playwright with Typescript interview questions

```
const browser = await chromium.launch({ headless: false });
const page = await browser.newPage();
await page.goto('https://example.com');
```

🟢 9. What is a locator?

✅ **Answer**

A locator is a **Playwright object used to find and interact with elements**.

💡 **Example**

```
const button = page.locator('#login');
await button.click();
```

🎯 **Why important**

- Auto-retry
- Stable
- Less flaky

🟢 10. Difference between locator and selector

Locator	Selector
Dynamic	Static
Auto-wait	No auto-wait
Recommended	Legacy

💡 **Example**

```
page.locator('#id')   // ✅ good
page.$('#id')         // ❌ old
```

🟢 11. What are different locator strategies?

✅ **Built-in locators**

Playwright with Typescript interview questions

page.getByRole('button')
page.getByText('Login')
page.getByLabel('Username')
page.getByPlaceholder('Enter name')
page.getByTestId('login-btn')

🔷 **Other locators**

page.locator('#id') // CSS
page.locator('//div') // XPath

🎯 **Best practice**

👉 Prefer getByRole() (most stable)

🟢 **12. What is auto-waiting?**

✅ **Answer**

Playwright automatically waits for elements before actions.

🔍 **It waits for:**

- Element visible
- Element enabled
- Element stable

💡 **Example**

await page.locator('#login').click();

👉 No need for Thread.sleep() ❌

🟢 **13. What are actionability checks?**

✅ **Answer**

Before performing an action, Playwright ensures:

1. Element is attached
2. Visible
3. Enabled

4. Stable (not moving)

🎯 **Interview line**

“Playwright ensures elements are actionable before interacting, reducing flaky tests.”

🟢 **14. What is default timeout and where to change it?**

✅ **Default**

👉 30 seconds

✅ **Change in config**

```
export default defineConfig({
  timeout: 60000
});
```

✅ **Change per action**

```
await page.click('#btn', { timeout: 5000 });
```

🟢 **15. What is playwright.config.ts?**

✅ **Answer**

This is the **main configuration file** that controls test execution.

🔍 **What it contains**

- baseURL
- timeout
- browser settings
- parallel execution
- reporters

💡 **Example**

Playwright with Typescript interview questions

```
import { defineConfig } from '@playwright/test';

export default defineConfig({
  use: {
    baseURL: 'https://example.com',
    headless: true
  },
  workers: 4,
  retries: 1
});
```

🎯 **Interview line**

“playwright.config.ts is the central configuration file used to manage environment, execution, and test behavior.”

Basic Actions questions PW

🟢 **1. How do you navigate to a URL?**

✅ **Answer**

You use page.goto() to open a URL.

💡 **Example**

```
await page.goto('https://example.com');
```

🔍 **With baseURL (best practice)**

```
await page.goto('/login');
```

🎯 **Interview line**

“Navigation is done using page.goto(), and in real frameworks we use baseURL to avoid hardcoding URLs.”

🟢 **2. What is baseURL and why is it used?**

✅ **Answer**

baseURL is defined in playwright.config.ts and acts as a **common root URL**.

Playwright with Typescript interview questions

💡 **Example (config)**

```
use: {
  baseURL: 'https://example.com'
}
```

💡 **Usage in test**

```
await page.goto('/login');
```

👉 Internally:

/login → https://example.com/login

🎯 **Why used**

- Avoid duplication
- Environment switching (QA/Prod)
- Cleaner code

🟢 **3. Difference between .fill() and .type()**

---

fill()	**type()**
Fast	Slow
Clears field	Types char by char
Used in automation	Simulates real typing

💡 **Example**

```
await page.fill('#username', 'admin');   // fast
await page.type('#username', 'admin');   // typing effect
```

🎯 **Interview line**

“fill is preferred for speed, while type is used when key events are required.”

🟢 **4. How do you perform click actions?**

Playwright with Typescript interview questions

💡 **Example**

await page.locator('#loginBtn').click();

🔍 **With options**

await page.click('#btn', { button: 'right' });

🎯 **Interview line**

“Click actions are performed using locator.click(), which includes auto-waiting.”

🟢 **5. How do you handle dropdowns?**

✅ **1. Standard dropdown (<select>)**

await page.selectOption('#dropdown', 'value1');

✅ **2. Custom dropdown**

await page.click('#dropdown');
await page.click('text=Option1');

🎯 **Interview line**

“We use selectOption for standard dropdowns and click-based locators for custom dropdowns.”

🟢 **6. How do you handle checkboxes and radio buttons?**

💡 **Example**

await page.check('#checkbox');
await page.uncheck('#checkbox');

Radio button

await page.check('#radioBtn');

Playwright with Typescript interview questions

🎯 **Interview line**

“Playwright provides check/uncheck methods which automatically ensure element state.”

🟢 **7. How do you handle alerts/dialogs?**

💡 **Example**

```
page.on('dialog', async dialog => {
  console.log(dialog.message());
  await dialog.accept();
});
```

🔷 **For dismiss**

```
await dialog.dismiss();
```

🎯 **Interview line**

“Playwright handles alerts using dialog event listeners.”

🟢 **8. How do you take screenshots?**

💡 **Full page**

```
await page.screenshot({ path: 'page.png' });
```

💡 **Element screenshot**

```
await page.locator('#logo').screenshot({ path: 'logo.png' });
```

🎯 **Interview line**

“Screenshots are useful for debugging and failure evidence.”

🟢 **9. How do you record video of execution?**

✅ **Enable in config**

```
use: {
  video: 'on'
}
```

🎯 **Options**

- on
- off
- retain-on-failure

🎯 **Interview line**

“Video recording helps analyze failures in CI environments.”

🟢 **10. How do you run tests from command line?**

💡 **Run all tests**

npx playwright test

💡 **Run specific file**

npx playwright test login.spec.ts

🟢 **11. How do you run tests in headed mode?**

💡 **Command**

npx playwright test --headed

🎯 **Why used**

- Debug visually
- Observe UI

🟢 **12. What is Playwright Inspector and how to use it?**

Playwright with Typescript interview questions

✅ **Answer**

Inspector is a **debugging tool** that pauses execution.

💡 **Run with**

npx playwright test --debug

🔍 **Features**

- Step-by-step execution
- Highlight elements
- Inspect locators

🎯 **Interview line**

“Inspector is used for debugging and analyzing test execution interactively.”

🟢 **13. What is codegen and how to use it?**

✅ **Answer**

Codegen is a **record-and-generate tool**.

💡 **Command**

npx playwright codegen https://example.com

🔍 **What it does**

- Records actions
- Generates Playwright code

🎯 **Interview line**

“Codegen is useful for quick script generation but not recommended for final framework code.”

🟢 **14. How do you perform hover action?**

Playwright with Typescript interview questions

💡 **Example**

```
await page.locator('#menu').hover();
```

🎯 **Use case**

- Menus
- Tooltips

🟢 **15. How do you check URL validation after navigation?**

💡 **Example**

```
await expect(page).toHaveURL('https://example.com/dashboard');
```

💡 **Partial match**

```
await expect(page).toHaveURL(/dashboard/);
```

🎯 **Interview line**

“URL validation ensures correct navigation and is done using expect assertions.”

Basic Scenarios in PW

🟢 **1. Automate login with valid credentials**

🎯 **Goal**

- Enter username/password
- Click login
- Verify successful navigation (URL or element)

💡 **Example**

```
import { test, expect } from '@playwright/test';

test('Login with valid credentials', async ({ page }) => {
  await page.goto('/login');
```

```
  await page.fill('#username', 'admin');
  await page.fill('#password', 'admin123');
  await page.click('#loginBtn');

  // Validation
  await expect(page).toHaveURL(/dashboard/);
});
```

🧠 **Interview explanation**

"After login, I validate using URL or dashboard element to confirm success."

🟢 **2. Automate login with invalid credentials**

🎯 **Goal**

- Enter wrong credentials
- Validate error message

💡 **Example**

```
test('Login with invalid credentials', async ({ page }) => {
  await page.goto('/login');

  await page.fill('#username', 'wrong');
  await page.fill('#password', 'wrong123');
  await page.click('#loginBtn');

  await expect(page.locator('.error-msg'))
    .toHaveText('Invalid username or password');
});
```

🧠 **Interview explanation**

"I validate error message visibility and content for negative scenarios."

🟢 **3. Automate form submission**

🎯 **Goal**

- Fill multiple fields
- Submit form
- Validate success

💡 **Example**

```
test('Form submission', async ({ page }) => {
  await page.goto('/form');

  await page.fill('#name', 'John');
  await page.fill('#email', 'john@test.com');
  await page.selectOption('#country', 'India');

  await page.click('#submitBtn');

  await expect(page.locator('.success'))
    .toHaveText('Form submitted successfully');
});
```

🟢 4. Automate checkbox selection

💡 Example

```
test('Checkbox selection', async ({ page }) => {
  await page.goto('/checkbox');

  await page.check('#agree');

  await expect(page.locator('#agree')).toBeChecked();
});
```

🟢 5. Automate radio button selection

💡 Example

```
test('Radio button selection', async ({ page }) => {
  await page.goto('/radio');

  await page.check('#male');

  await expect(page.locator('#male')).toBeChecked();
});
```

🟢 6. Automate file upload

🎯 Goal

Upload file using input element

💡 Example

```
test('File upload', async ({ page }) => {
  await page.goto('/upload');
```

```
  await page.setInputFiles('#fileUpload', 'tests/data/sample.pdf');

  await expect(page.locator('.upload-success'))
    .toHaveText('File uploaded');
});
```

🟢 7. Automate file download

🎯 Goal

Capture download event

💡 Example

```
test('File download', async ({ page }) => {
  await page.goto('/download');

  const downloadPromise = page.waitForEvent('download');
  await page.click('#downloadBtn');

  const download = await downloadPromise;

  console.log(await download.suggestedFilename());
});
```

🟢 8. Handle popup after clicking a button

🎯 Goal

Handle alert/dialog

💡 Example

```
test('Handle popup', async ({ page }) => {
  await page.goto('/popup');

  page.on('dialog', async dialog => {
    console.log(dialog.message());
    await dialog.accept();
  });

  await page.click('#popupBtn');
});
```

🟢 9. Automate search bar with suggestions

🎯 Goal

Playwright with Typescript interview questions

Type → wait for suggestions → select one

💡 **Example**

```
test('Search suggestions', async ({ page }) => {
  await page.goto('/search');

  await page.fill('#searchBox', 'iphone');

  await page.waitForSelector('.suggestion-item');

  await page.locator('.suggestion-item').first().click();

  await expect(page).toHaveURL(/iphone/);
});
```

🟢 **10. Handle element appearing after delay**

🎯 **Goal**

Wait dynamically (avoid hard wait)

💡 **Example**

```
test('Handle delayed element', async ({ page }) => {
  await page.goto('/delayed');

  const element = page.locator('#delayedBtn');

  await element.waitFor({ state: 'visible' });

  await element.click();
});
```

🧠 **Interview explanation**

“I use Playwright’s auto-wait or explicit waitFor instead of hard waits to handle dynamic elements.”

Playwright with Typescript interview questions

SECTION – B

MEDIUM (FRAMEWORK + LOGIC)

🔷 **TypeScript Intermediate**

🔷 **Playwright Intermediate**

🔷 **Framework Concepts**

🔷 **Medium Scenarios**

TypeScript Intermediate questions

🟡 **1. What are generics and how are they used in automation?**

✅ **Answer**

Generics allow you to write reusable functions/classes that work with multiple types while keeping type safety.

💡 **Example**

```
function getData<T>(data: T): T {
  return data;
}

let result = getData<string>("test");
```

🔍 **Real automation use**

👉 **Reading test data (JSON, API responses)**

```
async function getTestData<T>(file: string): Promise<T> {
  // read file and return typed data
  return JSON.parse('...');
}
```

🎯 **Interview line**

“Generics help build reusable and type-safe utilities in automation frameworks.”

🟡 **2. What are access modifiers (public, private, protected)?**

Playwright with Typescript interview questions

✅ **Answer**

They control visibility of class members.

Modifier	Access
public	Anywhere
private	Inside class only
protected	Class + child

💡 **Example**

```
class LoginPage {
  private username = '#user';

  public login() {}
}
```

🔍 **Real use**

👉 **Hide locators in POM**

🎯 **Interview line**

“Access modifiers ensure encapsulation and secure framework design.”

🟡 **3. What is function overloading?**

✅ **Answer**

Multiple function signatures with same name but different parameters.

💡 **Example**

```
function add(a: number, b: number): number;
function add(a: string, b: string): string;

function add(a: any, b: any) {
  return a + b;
}
```

Playwright with Typescript interview questions

🎯 **Use**

👉 **Flexible utility methods**

🟡 **4. What are optional parameters?**

✅ **Answer**

Parameters that may or may not be passed.

💡 **Example**

```
function createUser(name: string, age?: number) {
  console.log(name, age);
}
```

🎯 **Use**

👉 **Optional form fields**

🟡 **5. What are default parameters?**

✅ **Answer**

Assign default value if no argument is passed.

💡 **Example**

```
function greet(name = "Guest") {
  console.log(name);
}
```

🎯 **Use**

👉 **Default test data**

🟡 **6. What are arrow functions?**

✅ **Answer**

Short syntax for writing functions.

💡 **Example**

```
const add = (a: number, b: number) => a + b;
```

🎯 **Use**

👉 **Callbacks and utilities**

🟡 **7. What is module system in TypeScript?**

✅ **Answer**

Modules help organize code into separate files

💡 **Example**

```
// login.ts
export class Login {}

// test.ts
import { Login } from './login';
```

🎯 **Use**

👉 **Separate:**

- **pages**
- **utils**
- **tests**

🟡 **8. What is export and import?**

✅ **Answer**

Playwright with Typescript interview questions

Keyword	Purpose
export	Make available
import	Use in another file

💡 **Example**

```
export class User {}

import { User } from './User';
```

🎯 **Interview line**

“Export/import enables modular framework structure.”

🟡 **9. What is type assertion (as keyword)?**

✅ **Answer**

Tells TypeScript to treat a value as a specific type.

💡 **Example**

```
let value: unknown = "test";

let str = value as string;
```

⚠️ **Risk**

```
let val = null as string; // runtime crash
```

🎯 **Interview line**

“Type assertion bypasses type checking, so it should be used carefully.”

🟡 **10. What is strict mode in TypeScript?**

✅ **Answer**

Playwright with Typescript interview questions

Strict mode enforces strong type checking rules

💡 **Example (tsconfig)**

```
{
  "compilerOptions": {
    "strict": true
  }
}
```

🔍 **Includes:**

- **noImplicitAny**
- **strictNullChecks**

🎯 **Interview line**

“Strict mode improves code quality by enforcing safer type rules.”

🟡 **11. What are type guards and narrowing?**

✅ **Answer**

Used to check and narrow variable type

💡 **Example**

```
function print(value: string | number) {
  if (typeof value === "string") {
    console.log(value.toUpperCase());
  }
}
```

🎯 **Use**

👉 **Handling dynamic API data**

🟡 **12. What are utility types (Partial, Pick, Readonly)?**

Playwright with Typescript interview questions

✅ **Common utility types**

🔷 **Partial**

```
interface User {
  name: string;
  age: number;
}

let user: Partial<User> = { name: "Test" };
```

🔷 **Pick**

```
type UserName = Pick<User, 'name'>;
```

🔷 **Readonly**

```
const user: Readonly<User> = { name: "Test", age: 20 };
// cannot modify
```

🎯 **Use**

👉 **Flexible test data handling**

🟡 **13. What is inheritance in TypeScript?**

✅ **Answer**

Allows one class to reuse another class

💡 **Example**

```
class BasePage {
  navigate() {}
}

class LoginPage extends BasePage {}
```

🎯 **Use**

👉 **BasePage in framework**

Playwright with Typescript interview questions

🟡 **14. What are decorators?**

✅ **Answer**

Special functions that add metadata to classes/methods

💡 **Example**

```
function log(target: any) {
  console.log(target);
}

@log
class Test {}
```

🎯 **Use**

👉 **Framework-level enhancements (rare in Playwright)**

🟡 **15. What is declare keyword?**

✅ **Answer**

Used to tell TypeScript about external libraries

💡 **Example**

```
declare var $: any;
```

🎯 **Use**

👉 **When library has no type definitions**

Playwright with Typescript interview questions

Playwright Intermediate

🟡 1. Difference between locator() and page.$()

✅ Answer

locator()	page.$()
Recommended	Deprecated style
Auto-wait + retry	No auto-wait
Returns Locator	Returns ElementHandle
Stable	Can become stale

💡 Example

```
await page.locator('#login').click();  // ✅ best

const el = await page.$('#login');    // ❌ not recommended
await el?.click();
```

🎯 Interview line

“locator() is preferred because it auto-retries and avoids stale element issues.”

🟡 2. Why are locators preferred?

✅ Answer

- Built-in **auto-waiting**
- **Retry mechanism**
- More stable
- Avoid stale element exceptions

🎯 Real-world impact

👉 Reduces flaky tests

🟡 3. How do you handle iframes?

💡 Example

```
const frame = page.frameLocator('#frame');

await frame.locator('#btn').click();
```

🎯 Key point

👉 No switching required like Selenium

🟡 4. What is frameLocator()?

✅ Answer

Used to interact with elements inside iframe

💡 Example

```
await page.frameLocator('#payment-frame')
  .locator('#card')
  .fill('1234');
```

🟡 5. How do you handle Shadow DOM?

✅ Answer

Playwright automatically supports Shadow DOM

💡 **Example**

```
await page.locator('custom-element >> button').click();
```

🎯 **Key point**

👉 No extra handling required

🟡 **6. How do you handle multiple tabs/windows?**

💡 **Example**

```
const [newPage] = await Promise.all([
  context.waitForEvent('page'),
  page.click('#openTab')
]);

await newPage.waitForLoadState();
```

🎯 **Interview line**

“Use context.waitForEvent('page') to handle new tabs.”

🟡 **7. What is storageState?**

✅ **Answer**

Saves browser session (cookies + local storage)

💡 **Example**

```
await context.storageState({ path: 'auth.json' });
```

Playwright with Typescript interview questions

🟡 8. How do you reuse authentication?

💡 Step 1: Save login

```
await context.storageState({ path: 'auth.json' });
```

💡 Step 2: Use in config

```
use: {
  storageState: 'auth.json'
}
```

🎯 Benefit

👉 Skip login in all tests

🟡 9. How do you perform API testing?

💡 Example

```
const response = await request.get('/users');

expect(response.status()).toBe(200);
```

🎯 Use

👉 Backend validation + UI integration

🟡 10. How do you mock API responses (page.route())?

💡 Example

```
await page.route('**/api/users', route => {
  route.fulfill({
    status: 200,
    body: JSON.stringify({ name: 'Mock User' })
  });
});
```

🎯 Use

👉 Test edge cases without backend dependency

🟡 11. How do you upload files?

💡 Example

```
await page.setInputFiles('#file', 'tests/data/file.pdf');
```

🟡 12. What is trace viewer?

✅ Answer

A debugging tool showing:

- Steps
- Network logs
- DOM snapshot

💡 Enable

```
use: {
  trace: 'on'
}
```

Playwright with Typescript interview questions

🟡 13. How do you debug tests?

💡 Methods

npx playwright test --debug

Other ways:

- Inspector
- Console logs
- Trace viewer

🟡 14. How do you capture network logs?

💡 Example

```
page.on('request', req => console.log(req.url()));
page.on('response', res => console.log(res.status()));
```

🟡 15. How do you run tests in parallel?

💡 Config

workers: 4

🎯 Benefit

👉 Faster execution

🟡 16. What are workers?

✅ **Answer**

Workers are parallel threads executing tests

🎯 **Example**

- 4 workers → 4 tests run simultaneously

🟡 **17. How do you configure retries?**

💡 **Config**

retries: 2

🎯 **Use**

👉 Handle flaky tests

🟡 **18. How do you generate HTML reports?**

💡 **Config**

reporter: 'html'

💡 **Open report**

npx playwright show-report

🟡 **19. How do you configure multiple environments?**

💡 **Example**

```
use: {
  baseURL: process.env.BASE_URL
}
```

🎯 Run

```
BASE_URL=https://qa.app.com npx playwright test
```

🟡 20. How do you use environment variables?

💡 Using .env

Install:

```
npm install dotenv
```

💡 Example

```
require('dotenv').config();

process.env.USERNAME
```

🎯 Use

👉 Store:

- URLs
- Credentials
- Tokens

Playwright with Typescript interview questions

Framework Concepts

🟡 1. What is Page Object Model (POM)?

✅ Answer

POM is a design pattern where each page of the application is represented as a class.

👉 It separates:

- Test logic
- UI interaction logic

💡 Example

```
// pages/LoginPage.ts
import { Page } from '@playwright/test';

export class LoginPage {
  constructor(private page: Page) {}

  async login(username: string, password: string) {
    await this.page.fill('#user', username);
    await this.page.fill('#pass', password);
    await this.page.click('#loginBtn');
  }
}
```

💡 Test File

```
import { test } from '@playwright/test';
import { LoginPage } from '../pages/LoginPage';

test('Login Test', async ({ page }) => {
  const login = new LoginPage(page);
  await page.goto('/login');
  await login.login('admin', '1234');
});
```

🎯 Interview line

“POM separates test logic from UI logic, improving maintainability and reusability.”

Playwright with Typescript interview questions

🟡 2. Why do we use POM?

✅ Reasons

- Avoid duplicate code
- Centralized locators
- Easy maintenance
- Better readability

🎯 Real-time example

👉 If locator changes → update in one place

🎯 Interview line

“POM improves scalability and reduces maintenance effort in automation frameworks.”

🟡 3. How do you structure a Playwright framework?

📂 Real Structure

```
project/
|
 ├— tests/
 ├— pages/
 ├— fixtures/
 ├— utils/
 ├— test-data/
 ├— reports/
 ├— playwright.config.ts
```

🔍 Layer explanation

Folder	Purpose
tests	Test cases
pages	POM classes
fixtures	Dependency injection

Playwright with Typescript interview questions

Folder	Purpose
utils	Reusable logic
test-data	Input data
config	Execution control

🎯 Interview line

“We follow layered architecture with POM, fixtures, and utilities for scalability.”

🟡 4. What is package.json?

✅ Answer

It is the Node.js project configuration file

💡 Example

```
{
  "scripts": {
    "test": "npx playwright test"
  },
  "devDependencies": {
    "@playwright/test": "^1.40.0"
  }
}
```

🔍 Purpose

- Manage dependencies
- Define scripts
- Version control

🎯 Interview line

“package.json manages dependencies and execution commands.”

🟡 5. What is tsconfig.json in Playwright project?

Playwright with Typescript interview questions

✅ Answer

Defines how TypeScript is compiled

💡 Example

```
{
  "compilerOptions": {
    "target": "ES6",
    "strict": true,
    "module": "commonjs"
  }
}
```

🎯 Purpose

- Type safety
- Compilation rules
- Project consistency

🎯 Interview line

“tsconfig.json ensures consistent compilation and strict type checking.”

🟡 6. What are fixtures in Playwright? (REAL-TIME 🔥)

✅ Answer

Fixtures are reusable setup objects provided to tests.

🔍 Problem without fixtures

const login = new LoginPage(page); // repeated everywhere ❌

💡 With fixtures

```
// fixtures.ts
import { test as base } from '@playwright/test';
import { LoginPage } from '../pages/LoginPage';
```

Playwright with Typescript interview questions

```
export const test = base.extend({
  loginPage: async ({ page }, use) => {
    await use(new LoginPage(page));
  }
});
```

💡 Usage

```
test('Login', async ({ loginPage }) => {
  await loginPage.login('admin', '1234');
});
```

🎯 Real-time use

- Inject page objects
- Setup authentication
- Reuse resources

🎯 Interview line

“Fixtures provide dependency injection and reduce code duplication in tests.”

🟡 7. How do you create custom fixtures?

💡 Example

```
import { test as base } from '@playwright/test';

export const test = base.extend<{
  myFixture: string;
}>({
  myFixture: async ({}, use) => {
    await use('custom value');
  }
});
```

🎯 Use

- Inject test data
- Setup reusable objects

Playwright with Typescript interview questions

🟡 8. What is Base Page class?

✅ Answer

A common parent class for all pages

💡 Example

```
export class BasePage {
  constructor(public page: Page) {}

  async navigate(url: string) {
    await this.page.goto(url);
  }
}
```

💡 Child class

```
export class LoginPage extends BasePage {}
```

🎯 Use

- Common methods
- Reduce duplication

🎯 Interview line

“BasePage provides reusable methods across all page objects.”

🟡 9. What is utility/helper class?

✅ Answer

Contains reusable functions used across framework

💡 Example

```
export class Utils {
  static generateEmail() {
    return `test${Date.now()}@mail.com`;
```

```
 }
}
```

🎯 Use

- Random data
- API helpers
- Common logic

🟡 10. What is test data management?

✅ Answer

Handling input data for test cases

🔍 Methods

- JSON files
- Environment variables
- API-generated data

💡 Example

```
{
 "username": "admin",
 "password": "1234"
}
```

🎯 Usage

```
const data = require('../test-data/login.json');
await login.login(data.username, data.password);
```

🎯 Interview line

“Test data is managed separately to enable data-driven testing and avoid hardcoding.”

Playwright with Typescript interview questions

Medium Scenarios

🟡 1. Handle dynamic elements with changing IDs

🎯 Problem

IDs change on every load → #id_123, #id_456

✅ Approach

- Avoid ID/XPath with indexes
- Use **stable attributes / text / roles**

💡 Example

```
await page.getByRole('button', { name: 'Submit' }).click();
// or
await page.locator("//button[text()='Submit']").click();
```

🧠 Interview line

“I avoid dynamic attributes and use stable locators like text, role, or data-testid.”

🟡 2. Handle loading spinner with variable time

🎯 Problem

Spinner appears/disappears unpredictably

✅ Approach

Wait for spinner to disappear

💡 Example

```
await page.locator('.spinner').waitFor({ state: 'hidden' });
```

🟡 3. Handle table row actions dynamically

🎯 Problem

Click action in specific row

💡 Example

```
await page.getByRole('row')
  .filter({ hasText: 'user@example.com' })
  .getByRole('button', { name: 'Delete' })
  .click();
```

Playwright with Typescript interview questions

🟡 4. Handle custom dropdown (non-select)

🎯 Problem

Not <select> → built with

💡 Example

```
await page.click('#dropdown');
await page.click('text=Option 1');
```

🟡 5. Extract list of elements into array

💡 Example

```
const items = await page.locator('.list-item').allTextContents();
console.log(items);
```

🟡 6. Verify all links return 200 status (UI way)

💡 Example

```
const links = await page.locator('a').all();

for (const link of links) {
  const url = await link.getAttribute('href');
  if (url) {
    const response = await page.goto(url);
    console.log(response?.status());
  }
}
```

👉 (Not ideal for API—but acceptable UI-level validation)

🟡 7. Handle multiple browser tabs

💡 Example

```
const [newPage] = await Promise.all([
  page.context().waitForEvent('page'),
  page.click('#openTab')
]);

await newPage.waitForLoadState();
```

🟡 8. Automate drag and drop

💡 **Example**

```
await page.dragAndDrop('#source', '#target');
```

🟡 9. Automate hover-based menu

💡 **Example**

```
await page.locator('#menu').hover();
await page.click('text=Submenu');
```

🟡 10. Automate infinite scroll

💡 **Example**

```
await page.evaluate(async () => {
  for (let i = 0; i < 5; i++) {
    window.scrollBy(0, window.innerHeight);
    await new Promise(r => setTimeout(r, 1000));
  }
});
```

🟡 11. Automate pagination

💡 **Example**

```
while (await page.locator('text=Next').isVisible()) {
  // validate page data
  await page.click('text=Next');
}
```

🟡 12. Automate filters with multiple conditions

💡 **Example**

```
await page.selectOption('#category', 'Electronics');
await page.fill('#price', '1000');
await page.click('#applyFilter');

await expect(page.locator('.result')).toBeVisible();
```

🟡 13. Use storage state to skip login

💡 **Save state**

```
await context.storageState({ path: 'auth.json' });
```

Playwright with Typescript interview questions

💡 **Use in config**

```
use: {
  storageState: 'auth.json'
}
```

🧠 **Interview line**

“We reuse authentication using storage state to improve execution speed.”

🟡 **14. Wait for element covered by loader**

💡 **Example**

```
await page.locator('.loader').waitFor({ state: 'hidden' });
await page.click('#button');
```

🟡 **15. Retry flaky test scenario**

💡 **Config**

```
retries: 2
```

💡 **Better approach**

```
await expect(page.locator('#result')).toBeVisible();
```

🧠 **Interview line**

“Instead of relying only on retries, I fix flaky tests using proper waits and stable locators.”

🔴 **ADVANCED TYPESCRIPT**

🟣 **1. How do you implement advanced generics?**

✅ **Idea**

Generics with constraints + reusable utilities.

💡 **Example**

Playwright with Typescript interview questions

```
function getValue<T extends { id: number }>(obj: T): number {
  return obj.id;
}

const user = { id: 1, name: 'Test' };
getValue(user);
```

🧠 **Use in automation**

👉 Generic data readers / helpers

“I use constrained generics to ensure type-safe reusable utilities.”

🔵 **2. What are mapped types?**

✅ **Transform existing types**

💡 **Example**

```
type User = { name: string; age: number };

type ReadOnlyUser = {
  readonly [K in keyof User]: User[K];
};
```

🧠 **Use**

👉 Modify test data structures dynamically

🔵 **3. What are conditional types?**

💡 **Example**

```
type IsString<T> = T extends string ? true : false;

type A = IsString<string>; // true
```

🧠 **Use**

👉 Flexible typing in frameworks

🔵 **4. What are intersection types?**

💡 **Example**

```
type A = { name: string };
type B = { age: number };

type User = A & B;
```

Playwright with Typescript interview questions

🧠 **Use**

👉 Combine multiple interfaces (e.g., API + UI data)

🔵 **5. What are custom type guards?**

💡 **Example**

```
function isString(val: unknown): val is string {
  return typeof val === 'string';
}
```

🧠 **Use**

👉 Safe handling of dynamic values

🔵 **6. What is dependency injection in TypeScript?**

✅ **Idea**

Inject dependencies instead of creating inside class

💡 **Example**

```
class LoginPage {
  constructor(private page: Page) {}
}
```

🧠 **In Playwright**

👉 Fixtures = DI

🔵 **7. How do you design a Base class in POM?**

💡 **Example**

```
export class BasePage {
  constructor(protected page: Page) {}

  async navigate(url: string) {
    await this.page.goto(url);
  }

  async click(selector: string) {
```

```
  await this.page.locator(selector).click();
 }
}
```

🔵 **8. What are abstract classes?**

💡 **Example**

```
abstract class Base {
 abstract login(): void;
}
```

🧠 **Use**

👉 Force implementation in child classes

🔵 **9. What is declaration merging?**

💡 **Example**

```
interface User { name: string; }
interface User { age: number; }
```

👉 Merges automatically

🔵 **10. How do you manage large-scale type definitions?**

✅ **Approach**

- Separate types folder
- Reusable interfaces
- Avoid any

🔴 **ADVANCED PLAYWRIGHT**

🟠 **11. How Playwright internally handles auto-waiting?**

✅ **It checks:**

- Element attached
- Visible

Playwright with Typescript interview questions

- Enabled
- Stable

👉 Before every action

🧠 **Interview line**

“Playwright performs actionability checks before executing actions.”

🟠 **12. Hard wait vs smart wait**

Hard wait	Smart wait
waitForTimeout	locator-based
Unreliable	Reliable
Static	Dynamic

❌ **Bad**

```
await page.waitForTimeout(5000);
```

✅ **Good**

```
await page.locator('#btn').click();
```

🟠 **13. How Playwright communicates with browsers?**

✅ **Answer**

Uses:

- WebSocket
- Chrome DevTools Protocol (CDP)

👉 Faster than Selenium (HTTP)

🟠 **14. How do you implement soft assertions?**

💡 **Example**

```
await expect.soft(page.locator('#a')).toBeVisible();
await expect.soft(page.locator('#b')).toBeVisible();
```

👉 Test continues even if one fails

Playwright with Typescript interview questions

🟠 15. How do you integrate Allure reports?

💡 Steps

npm install allure-playwright

reporter: [['allure-playwright']]

🟠 16. How do you run tests in Docker?

💡 Example Dockerfile

```
FROM mcr.microsoft.com/playwright:v1.40.0
WORKDIR /app
COPY . .
RUN npm install
CMD ["npx", "playwright", "test"]
```

🟠 17. How do you integrate with Jenkins?

Pipeline flow:

1. Checkout
2. Install
3. Run tests
4. Publish report

🟠 18. How do you integrate with GitHub Actions?

💡 Example

```
name: Playwright Tests
on: [push]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - run: npm install
      - run: npx playwright test
```

🟠 19. How do you handle OAuth authentication?

UI approach (your focus)

Playwright with Typescript interview questions

- Login once
- Save storageState
- Reuse session

🟠 **20. Hybrid API + UI testing (UI perspective)**

👉 Example flow:

- Create user via API
- Validate in UI

(You can explain high-level only)

🟠 **21. How do you handle WebSocket testing?**

👉 Mostly advanced → listen events

page.on('websocket', ws => console.log(ws.url()));

🟠 **22. How do you intercept and abort network requests?**

💡 **Example**

await page.route('**/*.png', route => route.abort());

🟠 **23. How do you mock geolocation/timezone?**

💡 **Example**

use: {
  geolocation: { latitude: 12.97, longitude: 77.59 },
  permissions: ['geolocation']
}

🟠 **24. How do you use page.evaluate()?**

💡 **Example**

const title = await page.evaluate(() => document.title);

🧠 **Use**

Playwright with Typescript interview questions

👉 Execute JS inside browser

🟠 **25. What is visual regression testing?**

✅ **Answer**

Compare UI screenshots to detect visual changes

💡 **Example**

```
await expect(page).toHaveScreenshot();
```

//Grouping//Hooks//tagging//logs

🟦 **1. GROUPING TESTS (test.describe)**

✅ **What is it?**

Used to **group related test cases together**

💡 **Example**

```
import { test, expect } from '@playwright/test';

test.describe('Login Tests', () => {

  test('Valid login', async ({ page }) => {
    // test code
  });

  test('Invalid login', async ({ page }) => {
    // test code
  });

});
```

🧠 **Why used?**

- Better organization
- Shared setup (hooks)

Playwright with Typescript interview questions

- Logical grouping

🎯 **Interview line**

“test.describe is used to group related test cases and manage them together.”

🟦 **2. SKIP TESTS**

✅ **Skip single test**

```
test.skip('Skip this test', async ({ page }) => {});
```

✅ **Skip based on condition**

```
test.skip(process.env.ENV === 'prod', 'Skipping in prod');
```

✅ **Skip group**

```
test.describe.skip('Skip all tests', () => {});
```

🎯 **Use case**

- Feature not ready
- Environment issues

🟦 **3. HOOKS (VERY IMPORTANT 🔥)**

🔷 **Types of hooks**

Hook	Purpose
beforeAll	Runs once before all tests
beforeEach	Runs before each test
afterEach	Runs after each test
afterAll	Runs once after all tests

💡 **Example**

Playwright with Typescript interview questions

```
test.beforeEach(async ({ page }) => {
  await page.goto('/login');
});

test.afterEach(async ({ page }) => {
  console.log('Test completed');
});
```

🧠 **Real-time use**

- Login setup
- Cleanup
- Test initialization

🎯 **Interview line**

“Hooks are used for setup and teardown to avoid duplication.”

🟦 **4. TAGGING (@tags) + --grep**

✅ **Tagging tests**

```
test('Login test @smoke', async ({ page }) => {});
test('Checkout test @regression', async ({ page }) => {});
```

✅ **Run specific tags**

```
npx playwright test --grep @smoke
```

✅ **Exclude tag**

```
npx playwright test --grep-invert @slow
```

🧠 **Use case**

- Smoke suite
- Regression suite
- Sanity runs

Playwright with Typescript interview questions

🎯 **Interview line**

“Tags help in selective test execution using grep options.”

🟦 **5. LOGGING (IMPORTANT)**

✅ **Basic logs**

```
console.log('Step executed');
```

✅ **Better logging (structured)**

```
test('Example', async ({ page }) => {
  console.log('Navigating to page');
  await page.goto('/login');
});
```

✅ **Playwright logs (trace + debug)**

```
npx playwright test --debug
```

🧠 **Real-time logging**

- Debug failures
- Track execution
- CI visibility

🟦 **6. ONLY (Run specific test)**

```
test.only('Run only this test', async ({ page }) => {});
```

⚠️ **Important**

👉 Don’t push .only to repo ❌

🟦 **7. PARALLEL GROUPING**

```
test.describe.parallel('Parallel tests', () => {
  test('Test1', async () => {});
  test('Test2', async () => {});
});
```

🟦 8. SERIAL EXECUTION

```
test.describe.serial('Sequential tests', () => {
  test('Step1', async () => {});
  test('Step2', async () => {});
});
```

🟦 9. TIMEOUT PER TEST

```
test('Slow test', async ({ page }) => {
  // test
}).setTimeout(60000);
```

🟦 10. RETRY AT TEST LEVEL

```
test('Retry test', async ({ page }) => {
}).retry(2);
```

🟦 REAL-TIME INTERVIEW ANSWER (IMPORTANT 🔥)

If asked:

👉 “How do you organize and control execution in Playwright?”

Say:

“We use test.describe for grouping, hooks for setup and teardown, tagging with grep for selective execution, and features like skip, only, retries, and parallel execution to control test runs efficiently.”

Playwright with Typescript interview questions

//Real time challenges in playwright

🟦 1. Real-Time Flow (Very Important)

In real automation:

UI → Download PDF → Save file → Read PDF → Validate content

👉 Responsibility split:

Layer	Responsibility
Playwright	Download file
Utils	Read & extract PDF content
Test	Validate data

🟦 2. Step 1: Download PDF using Playwright

💡 Example

```
import { test, expect } from '@playwright/test';

test('Download PDF', async ({ page }) => {
  await page.goto('/reports');

  const downloadPromise = page.waitForEvent('download');

  await page.click('#downloadPdf');

  const download = await downloadPromise;

  const filePath = await download.path();

  console.log(filePath);
});
```

🧠 Important

- download.path() → gives file location
- File is temporarily stored

🟦 3. Step 2: Install PDF Reader Library

Playwright **cannot read PDF content directly**

👉 Use Node library:

npm install pdf-parse

🟦 4. Step 3: Create Utility for PDF Extraction

📂 utils/pdfUtils.ts

```
import fs from 'fs';
import pdf from 'pdf-parse';

export async function readPDF(filePath: string): Promise<string> {
  const buffer = fs.readFileSync(filePath);
  const data = await pdf(buffer);

  return data.text;
}
```

🧠 Why in utils?

👉 Because:

- Reusable
- Clean separation
- Keeps test code simple

🟦 5. Step 4: Use Utility in Test

💡 Example

```
import { test, expect } from '@playwright/test';
import { readPDF } from '../utils/pdfUtils';

test('Validate PDF content', async ({ page }) => {
  await page.goto('/reports');

  const downloadPromise = page.waitForEvent('download');
```

```
  await page.click('#downloadPdf');

  const download = await downloadPromise;

  const filePath = await download.path();

  const text = await readPDF(filePath!);

  expect(text).toContain('Invoice Number');
});
```

🟦 6. Real-Time Improvements (VERY IMPORTANT 🔥)

✅ Save file to specific location

```
const filePath = `downloads/${download.suggestedFilename()}`;
await download.saveAs(filePath);
```

✅ Validate multiple fields

```
expect(text).toContain('Total Amount');
expect(text).toContain('Customer Name');
```

✅ Clean up file after test

```
fs.unlinkSync(filePath);
```

🟦 7. Interview Answer (Use This)

👉 If asked:

“How do you validate PDF in Playwright?”

Say:

“Playwright is used to download the PDF file using page.waitForEvent('download'). The file is then processed using a Node.js library like pdf-parse inside a utility class. The extracted content is validated in the test. This approach maintains separation of concerns and improves reusability.”

🟦 8. Key Takeaways

✓ Playwright → download only
✓ Utils → extract data
✓ Test → validation

Playwright with Typescript interview questions

Outlook instead of test account opening original account -

🟦 **First: Why your original Outlook account is opening?**

👉 This happens because:

- Browser already has **saved session/cookies**
- Playwright is **reusing that session indirectly (headed mode / system browser confusion)**

👉 But important correction:

❌ Playwright does NOT use your Chrome profile by default
✓ It creates a **fresh browser context every time**

So if your personal account is opening, it means:

👉 Either:

- You logged in during previous test and reused storage
- Or you are using saved storageState
- Or you are testing in a way that keeps session alive

🟦 **Key Concept: BrowserContext = Incognito (VERY IMPORTANT 🔥)**

✅ **What is BrowserContext?**

👉 Think like this:

Browser → Context → Page

- **Browser** = Chrome
- **Context** = Incognito window
- **Page** = Tab

💡 **Example**

```
const context1 = await browser.newContext(); // Incognito 1
const page1 = await context1.newPage();

const context2 = await browser.newContext(); // Incognito 2
const page2 = await context2.newPage();
```

👉 Both contexts:

- Have **separate cookies**
- Have **separate sessions**

Playwright with Typescript interview questions

- Do NOT share login

🟦 **Your Problem (Real Explanation)**

❌ **What you are doing now:**

1. Login to Outlook
2. Session gets stored
3. Next run → same account auto opens

✅ **What you SHOULD do:**

👉 Always create a **fresh context**

🟦 **Solution 1: Use fresh context (Recommended)**

💡 **Example**

```
import { test } from '@playwright/test';

test('Outlook login with fresh session', async ({ browser }) => {

  const context = await browser.newContext();  // 🔥 new incognito
  const page = await context.newPage();

  await page.goto('https://outlook.live.com');

  // Now login manually
});
```

👉 This ensures:
✓ No previous login
✓ Clean session
✓ No personal account

🟦 **Solution 2: Disable stored authentication (if using storageState)**

Check your config:

```
use: {
  storageState: 'auth.json'   // ❌ this reuses login
}
```

👉 Remove this if you want fresh login

🟦 Solution 3: Explicit logout (less reliable)

```
await page.click('#logout');
```

👉 Not preferred in real automation

🟦 How to Handle Outlook OTP Scenario (REAL-TIME 🔥)

🔁 Flow:

Login App → Trigger OTP → Open Outlook → Read OTP → Validate

💡 Example Approach

```
test('Read OTP from Outlook', async ({ browser }) => {

  const context = await browser.newContext();  // fresh session
  const page = await context.newPage();

  await page.goto('https://outlook.live.com');

  // login with test account
  await page.fill('#login', 'test@outlook.com');
  await page.fill('#password', 'password');

  // open inbox
  await page.click('text=Inbox');

  // get OTP text
  const otpText = await page.getByText('Your code').textContent();

  console.log(otpText);
});
```

🟦 Better Approach (REAL PROJECT LEVEL 🔥🔥)

Playwright with Typescript interview questions

👉 Don’t automate UI for Outlook (bad practice ❌)

👉 Instead:

✓ Use **Microsoft Graph API / IMAP**
✓ Fetch email directly
✓ Extract OTP

🎯 **Why?**

UI Automation	API Approach
Slow	Fast
Flaky	Stable
UI dependent	Backend reliable

🟦 **Interview Answer (IMPORTANT)**

If interviewer asks:

👉 “How do you handle OTP from Outlook?”

Say:

“Initially we automated using UI by logging into Outlook and extracting OTP using locators. However, this approach is flaky and slow. In real projects, we prefer using APIs like Microsoft Graph or IMAP to fetch email content directly. Also, we ensure isolation using BrowserContext to avoid session conflicts.”

🟦 **Key Takeaways**

✓ BrowserContext = Incognito
✓ Always use fresh context for clean login
✓ Avoid UI email automation in real projects
✓ Prefer API-based email reading